Architecture of Complex Systems

Week 1: Systems Thinking

Project Portfolio

Name

Martin Santiago Benavides Ardila

Copyright © 2024. Massachusetts Institute of Technology. All rights reserved.

Instructions

Before you begin, you should save your Project Portfolio on your local drive. We recommend the following format:

Lastname_Firstname_Course1_Week1

Please note: You will not be able to re-download your file after submission; therefore, please keep this file in a central location for future reference.

The project deliverable is an **individual submission**. A scoring rubric can be downloaded from the "Project Instructions" page for this week.

You will be self-assessing your work. If you have any questions, feel free to start a thread in the Discussion Forum.

Note: edX has a 10MB file size limit for document submission. If you have selected large image(s), you may need to resize before submitting, OR you may simply include a web URL for the image in the image location. Be sure to submit your assignment at least one hour before the deadline to provide time for troubleshooting.

Once the deadline passes, you will not be able to upload the document and therefore will not be able to submit and complete the assignment.

Copyright © 2024. Massachusetts Institute of Technology. All rights reserved.

Week 1 Project

Overview

In this first project activity, you will apply the four tasks of systems thinking to a real-life "system." You will have the option to choose from five different system types, which are shown on the next slide.

We recommend that you do some brief research on each one before choosing; however, if one is particularly relevant to your occupation, feel free to simply select that. You will then follow the steps which appear in the text box to the right to complete this activity.

REQUIRED STEPS:

Step 1: Research and select your project system.

Step 2: Identify your system's FORM and FUNCTION.

Step 3: Identify the primary ENTITIES in your system.

Step 4: Identify RELATIONSHIPS in your system.

Step 5: Predict your system's EMERGENCE.

Step 6: Develop system DECOMPOSITION.

Step 7: Review and submit your project.

Copyright © 2024. Massachusetts Institute of Technology. All rights reserved.

SYSTEM DETAILS

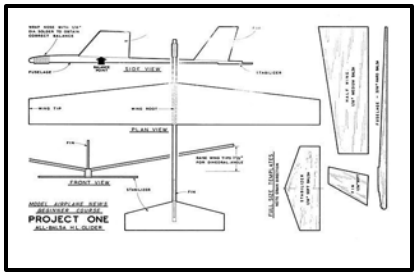
The following details pertain to the systems described on this slide and provide general guidance. You may use a variation of one of the five systems as long as it maintains the same level of complexity as below. For instance, you may not choose a half bit adder instead of a full bit adder. You should include an image of the system that you use.

- Balsa wood glider: A beginner-level model hand-launched glider airplane composed of the following: fuselage, rear stabilizer, fin, wings, and front counterbalance weight.
- Crystal radio: Simple early form of a radio receiver circuit composed of the following: variable capacitor, aerial terminal, inductor coil, diode, resistor, earplug output, and a ground terminal.
- Prime number search code: C void function that prints all the prime numbers less than or equal to the number given in variable *n*. The function is composed of: variable *n*, array of length *n* named “primes,” for loop that flags all items in the array as prime numbers, for loop that computes for the prime numbers in the array, for loop that prints out to the prime numbers of the array.
- Simple refracting telescope: Optical telescope based on two lenses. The system is composed of a focal tube, objective lens, eyepiece, eye lens, focus knob, three tripod legs, focal tube mount, and tripod flange.
- One-bit full adder: Digital circuit that adds three one-bit numbers. This system is composed of two XOR gates, two AND gates, and one OR gate.

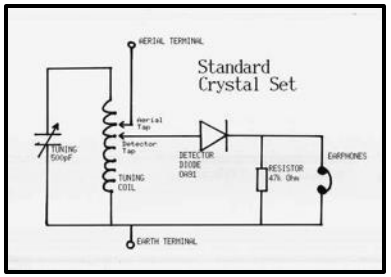
Copyright © 2024. Massachusetts Institute of Technology. All rights reserved.

Step 1: RESEARCH & SELECT YOUR SYSTEM

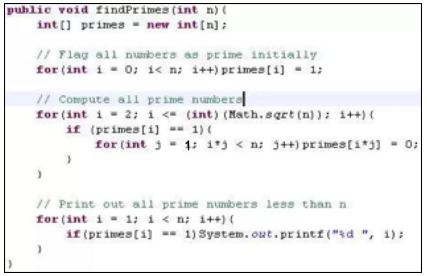
My system choice: Prime Number Search Code



Balsa Wood Glider

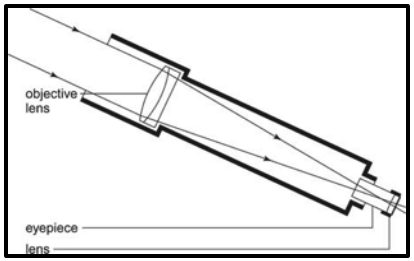


Crystal Radio

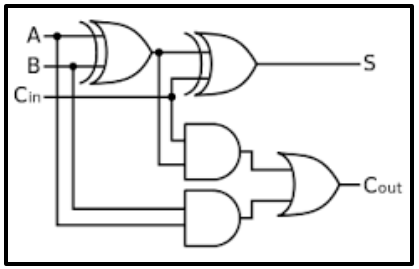


```
public void findPrimes(int n){
    int[] primes = new int[n];

    // Flag all numbers as prime initially
    for(int i = 0; i< n; i++)primes[i] = 1;

    // Compute all prime numbers
    for(int i = 2; i <= (int)(Math.sqrt(n)); i++){
        if (primes[i] == 1){
            for(int j = 1; i*j < n; j++)primes[i*j] = 0;
        }
    }

    // Print out all prime numbers less than n
    for(int i = 1; i < n; i++){
        if(primes[i] == 1)System.out.printf("%d ", i);
    }
}
```

Prime Number Search Code



Simple Refracting Telescope



1-bit Adder

Copyright © 2024. Massachusetts Institute of Technology. All rights reserved.

Step 2: IDENTIFY SYSTEM FORM & FUNCTION

Insert image you sourced representing your selected system in the box on the left side of the slide below. Then indicate the examples of FORM and FUNCTION that you've identified in the field on the right below.

Note: edX has a 10MB file size limit for document submission. If you have selected large image(s), you may need to resize before submitting, OR you may simply include a web URL for the image in the image location.

System Diagram/Schematic

```
public void findPrimes(int n){
    int[] primes = new int[n];

    // Flag all numbers as prime initially
    for(int i = 0; i< n; i++)primes[i] = 1;

    // Compute all prime numbers
    for(int i = 2; i <= (int)(Math.sqrt(n)); i++){
        if (primes[i] == 1){
            for(int j = 1; i*j < n; j++)primes[i*j] = 0;
        }
    }

    // Print out all prime numbers less than n
    for(int i = 1; i < n; i++){
        if(primes[i] == 1)System.out.printf("%d ", i);
    }
}
```

Primary System FORM:

C Function findPrimes(int n). Code structure: listing the variable, the array and 3 loops for logic program

Primary System FUNCTION:

Identifies all prime numbers <= n (given positive integer)

Describe what is outside the system boundary, in terms of forms outside the boundary and functions outside the boundary.

The caller that supplies n, the standard output stream or console where results are printed, the C runtime and compiler, and the hardware executing the code. Those are forms and functions that exist outside the function itself but interact with it and helps it works.

Copyright © 2024. Massachusetts Institute of Technology. All rights reserved.

Step 3: IDENTIFY SYSTEM ENTITIES

Copy the image you inserted into Step 2 here. Then highlight or circle the different entities in your system and indicate the respective form and function for two of them in the fields at right.

Note: edX has a 10MB file size limit for document submission. If you have selected large image(s), you may need to resize before submitting, OR you may simply include a web URL for the image in the image location.

System Diagram/Schematic

```
public void findPrimes(int n){
    int[] primes = new int[n];

    // Flag all numbers as prime initially
    for(int i = 0; i< n; i++)primes[i] = 1;

    // Compute all prime numbers
    for(int i = 2; i <= (int)(Math.sqrt(n)); i++){
        if (primes[i] == 1){
            for(int j = 1; i*j < n; j++)primes[i*j] = 0;
        }
    }

    // Print out all prime numbers less than n
    for(int i = 1; i < n; i++){
        if(primes[i] == 1)System.out.printf("%d ", i);
    }
}
```

System Entity 1:

primes[]

Form: Integer array

Function: Store the state of every integer

System Entity 2:

Nested loop

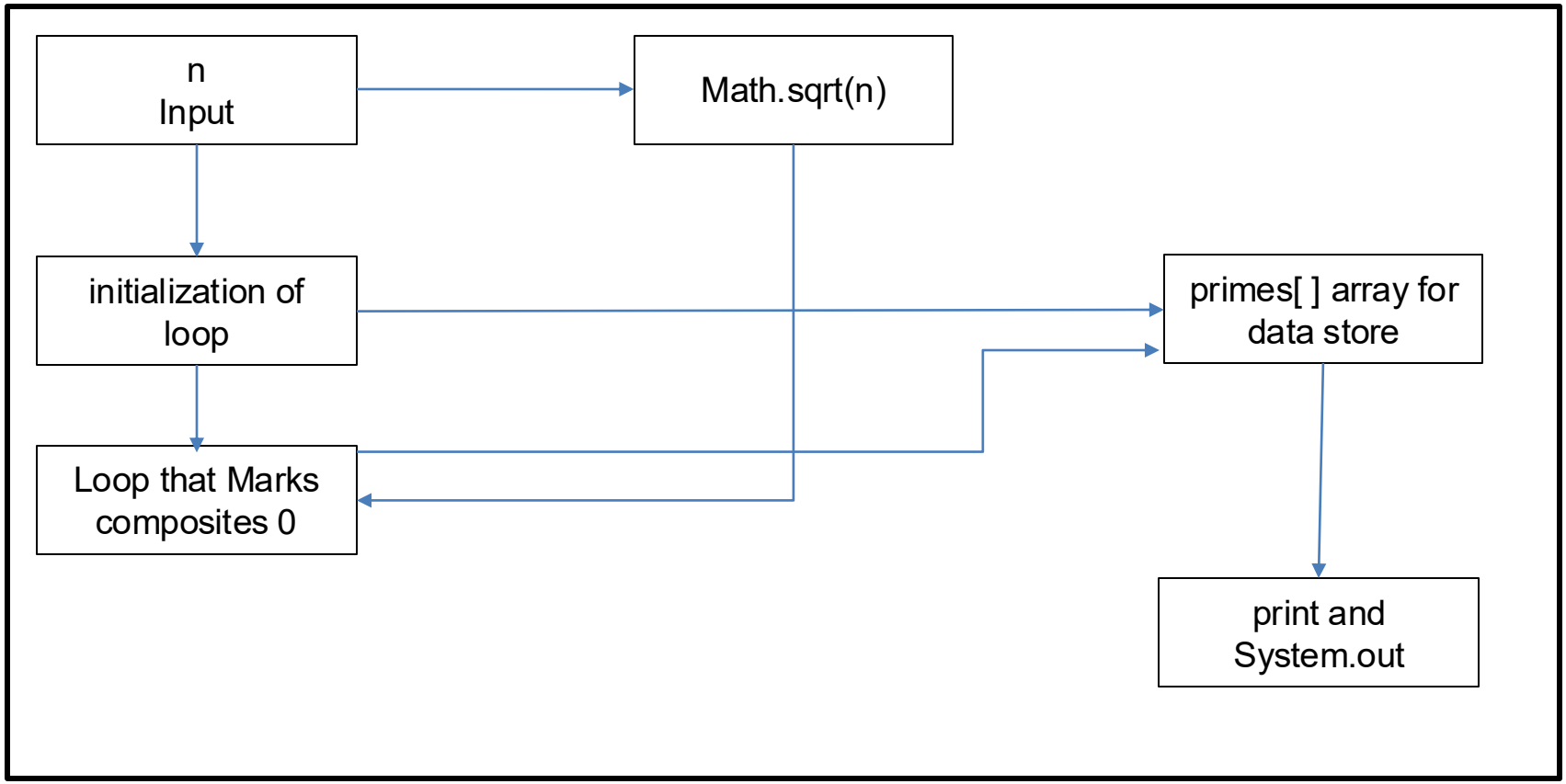
Form: ”for” loop structure

Function: Compute and does the print-finding work

Copyright © 2024. Massachusetts Institute of Technology. All rights reserved.

Step 4: IDENTIFY SYSTEM RELATIONSHIPS

Replace the names of Entities 1 and 2 in the diagram below with the first two entities you identified on the previous step. Identify at least 4 more entities in your system. Then, use the connectors to define the relationships between your entities. You may copy the objects you use more than once and delete any you don't use.



Copyright © 2024. Massachusetts Institute of Technology. All rights reserved.

Step 5: PREDICT SYSTEM EMERGENCE

For this step, you will predict two types of emergence in your system: intended and unintended, including emergence failures. Write a brief description of each emergent behavior of your system, and then explain how it occurred by describing it functional emergence.
You may also upload an image for each emergence type if you prefer. Please remember the file size limit and resize or paste the image URL instead, as needed.

Intended Emergence

A complete list of numbers less than n

Functional Interaction

The init loop flags all indices as prie -> loop multiplies of each remaining prime up to sqrt(n) in the shared primes[ ] array -> print loop outputs the surviving flags.

(optional) **Image**

Insert image here.

Unintended Emergence

Indices 0 and 1 are printed as primes

Functional Interaction

The init loop writes to the full range [0,n-1] -> loop operates from i=2 onward -> 0 and 1 untouched for the print loop to output.

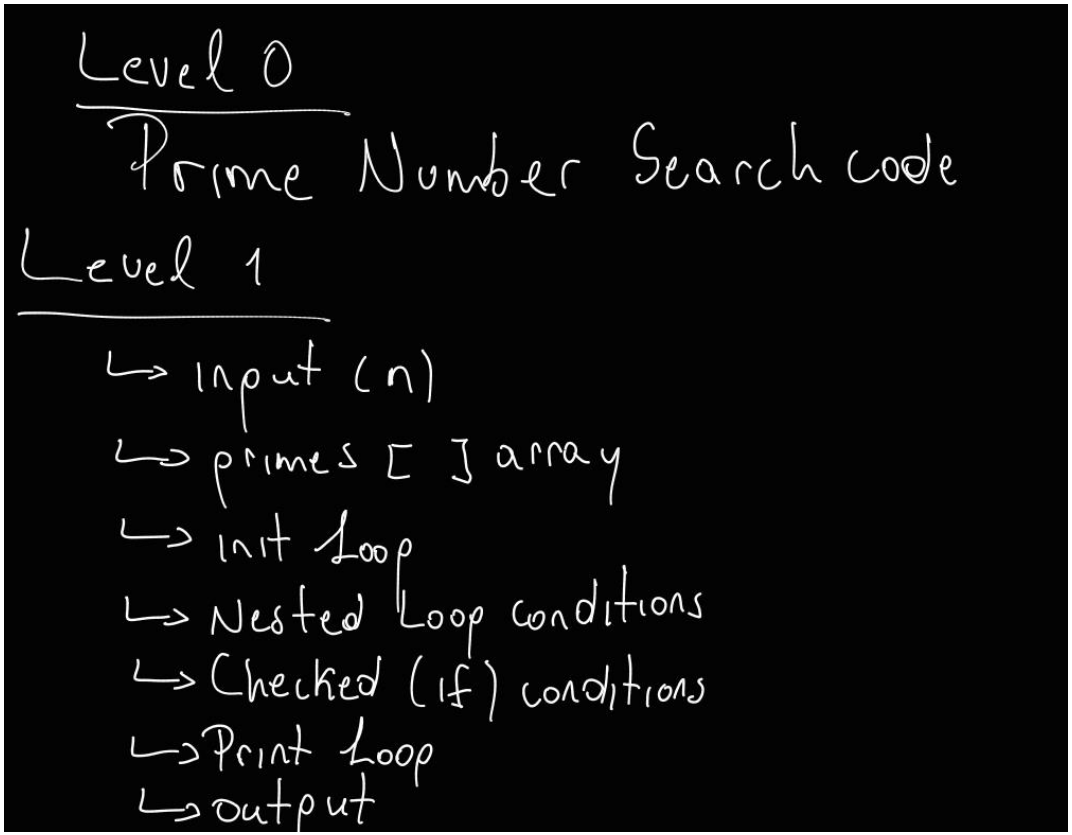
(optional) **Image**

Insert image here.

Copyright © 2024. Massachusetts Institute of Technology. All rights reserved.

Step 6: DEVELOP SYSTEM DECOMPOSITION

For your last step, develop a Level One system decomposition. Draw a decompositional view of your system that includes Level Zero and Level One.



Copyright © 2024. Massachusetts Institute of Technology. All rights reserved.

STEP 7: REVIEW & SUBMIT PROJECT

- Submit and self assess your completed Week 1 Project Portfolio file Note: The maximum file size that can be submitted is 10MB.
  - A sample project submission and scoring rubric can be downloaded from the “Project Instructions” page for this week.
  - Please remember that there are two parts to this assignment: submission and self assessment. Please be sure to provide enough time to complete both parts.

Copyright © 2024. Massachusetts Institute of Technology. All rights reserved.